

MySQL Database Assignment & Study Notes

1. What is MySQL?

MySQL is an open-source relational database management system. It is used to store, organize, manage and retrieve data for websites, mobile applications and business systems.

Example: A restaurant application can use MySQL to store menu categories, products, customers, orders and order items.

2. Important Database Terms

Term	Simple Meaning
Database	A collection of related information.
Table	Stores information in rows and columns.
Row / Record	One complete item, such as one customer.
Column / Field	One type of information, such as customer_name.
Primary Key	A unique ID used to identify each row.
Foreign Key	A column used to connect one table to another.
SQL	The language used to communicate with a relational database.
Query	An SQL command used to read or change data.
Constraint	A rule that helps keep data correct.
Index	A database structure that can make searches faster.

3. Database Design

Our simple database relationship is:

```
categories  1 ---- many  products
customers   1 ---- many  orders
orders      1 ---- many  order_items
products    1 ---- many  order_items
```

Explanation: One category can have many products. One customer can place many orders. One order can contain many order items, and each order item points to a product.

4. Create the Database

```
CREATE DATABASE restaurant_assignment_db;
```

Explanation: CREATE DATABASE creates a new database.

5. Create the Tables

5.1 Categories Table

```
CREATE TABLE categories (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Explanation: id is the unique category ID. AUTO_INCREMENT creates the next ID automatically. name is required because of NOT NULL. created_at stores the creation date and time automatically.

5.2 Products Table

```
CREATE TABLE products (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  category_id INT NOT NULL,  
  name VARCHAR(150) NOT NULL,  
  description VARCHAR(255),  
  price DECIMAL(10,2) NOT NULL,  
  is_active BOOLEAN DEFAULT TRUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (category_id) REFERENCES categories(id)  
);
```

Explanation: category_id connects each product to a category. DECIMAL(10,2) is suitable for money values. The foreign key prevents a product from pointing to a category that does not exist.

5.3 Customers Table

```
CREATE TABLE customers (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(120) NOT NULL,  
  email VARCHAR(150) UNIQUE,  
  phone VARCHAR(30),  
  city VARCHAR(100),  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Explanation: The UNIQUE rule prevents two customers from using the same email address.

5.4 Orders Table

```
CREATE TABLE orders (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  customer_id INT NOT NULL,  
  order_date DATETIME DEFAULT CURRENT_TIMESTAMP,  
  status VARCHAR(30) DEFAULT 'Pending',  
  total_amount DECIMAL(10,2) DEFAULT 0.00,  
  FOREIGN KEY (customer_id) REFERENCES customers(id)  
);
```

Explanation: Each order belongs to one customer. status stores values such as Pending, Confirmed or Completed.

5.5 Order Items Table

```
CREATE TABLE order_items (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  order_id INT NOT NULL,  
  product_id INT NOT NULL,  
  quantity INT NOT NULL,  
  unit_price DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (order_id) REFERENCES orders(id),  
  FOREIGN KEY (product_id) REFERENCES products(id)  
);
```

Explanation: This table stores the products inside each order. quantity tells us how many items were ordered, and unit_price stores the product price used for that order.

6. Insert Sample Data

6.1 Insert Categories

```
INSERT INTO categories (name)  
VALUES  
( 'Breakfast'),  
( 'Main Course'),  
( 'Drinks'),  
( 'Desserts'),  
( 'Salads');
```

Explanation: This inserts five category records at the same time.

6.2 Insert Products

```
INSERT INTO products (category_id, name, description, price)  
VALUES  
(1, 'Egg Breakfast', 'Eggs with bread and vegetables', 28.00),  
(1, 'Pancakes', 'Pancakes with honey', 24.00),  
(2, 'Chicken Burger', 'Grilled chicken burger', 35.00),  
(2, 'Beef Burger', 'Beef burger with cheese', 42.00),  
(2, 'Chicken Pasta', 'Creamy chicken pasta', 48.00),  
(3, 'Orange Juice', 'Fresh orange juice', 18.00),  
(3, 'Coffee', 'Hot black coffee', 14.00),  
(3, 'Water', 'Mineral water', 6.00),  
(4, 'Chocolate Cake', 'Chocolate cake slice', 25.00),  
(4, 'Ice Cream', 'Vanilla ice cream', 20.00),  
(5, 'Caesar Salad', 'Chicken Caesar salad', 32.00),  
(5, 'Green Salad', 'Fresh mixed green salad', 22.00);
```

Explanation: Each product uses category_id to connect it to a category.

6.3 Insert Customers

```
INSERT INTO customers (name, email, phone, city)  
VALUES  
( 'Ahmed Ali', 'ahmed@example.com', '0501111111', 'Riyadh'),  
( 'Sara Khan', 'sara@example.com', '0502222222', 'Jeddah'),  
( 'Mohammed Saeed', 'mohammed@example.com', '0503333333', 'Dammam'),
```

```
('Ali Hassan', 'ali@example.com', '0504444444', 'Riyadh'),  
( 'Fatima Noor', 'fatima@example.com', '0505555555', 'Khobar');
```

Explanation: This gives us sample customers that we can use when creating orders.

6.4 Insert Orders

```
INSERT INTO orders (customer_id, status, total_amount)  
VALUES  
(1, 'Completed', 88.00),  
(2, 'Pending', 66.00),  
(1, 'Confirmed', 50.00);
```

Explanation: Customer 1 has two orders. This demonstrates a one-to-many relationship.

6.5 Insert Order Items

```
INSERT INTO order_items (order_id, product_id, quantity, unit_price)  
VALUES  
(1, 3, 2, 35.00),  
(1, 6, 1, 18.00),  
(2, 4, 1, 42.00),  
(2, 2, 1, 24.00),  
(3, 9, 2, 25.00);
```

Explanation: Order 1 contains two Chicken Burgers and one Orange Juice. The total is $2 \times 35 + 18 = 88$.

7. SELECT - Read Data

```
SELECT * FROM products;
```

Explanation: The asterisk (*) means return all columns from all product rows.

```
SELECT name, price FROM products;
```

Explanation: This returns only the name and price columns.

8. WHERE - Filter Data

```
SELECT * FROM products WHERE price > 30;
```

Explanation: Returns only products with a price greater than 30.

```
SELECT * FROM customers WHERE city = 'Riyadh';
```

Explanation: Returns customers whose city is Riyadh.

9. AND / OR

```
SELECT * FROM products  
WHERE price >= 20 AND price <= 40;
```

Explanation: AND means both conditions must be true.

```
SELECT * FROM customers
WHERE city = 'Riyadh' OR city = 'Jeddah';
```

Explanation: OR means at least one condition must be true.

10. LIKE - Search Text

```
SELECT * FROM products WHERE name LIKE '%Burger%';
```

Explanation: The % symbol means any text can appear before or after Burger. This query finds product names containing Burger.

11. ORDER BY - Sort Results

```
SELECT * FROM products ORDER BY price ASC;
```

Explanation: ASC sorts from lowest price to highest price.

```
SELECT * FROM products ORDER BY price DESC;
```

Explanation: DESC sorts from highest price to lowest price.

12. LIMIT - Return a Small Number of Rows

```
SELECT * FROM products ORDER BY price DESC LIMIT 5;
```

Explanation: This returns only the five most expensive products.

13. UPDATE - Change Data

```
UPDATE products
SET price = 38.00
WHERE id = 3;
```

Explanation: This changes the price of product ID 3. Always use WHERE carefully. Without WHERE, every product could be updated.

14. DELETE - Remove Data

```
INSERT INTO products (category_id, name, price)
VALUES (3, 'Test Drink', 1.00);

DELETE FROM products
WHERE name = 'Test Drink';
```

Explanation: The first query creates a test record. The DELETE query removes only that test record. Always check your WHERE condition before deleting.

15. INNER JOIN - Combine Related Tables

```
SELECT
    p.id,
    p.name AS product,
    c.name AS category,
    p.price
FROM products p
INNER JOIN categories c
    ON p.category_id = c.id;
```

Explanation: The JOIN connects products with categories. p and c are short aliases that make the query easier to read.

16. Order Report Using Multiple JOINS

```
SELECT
    o.id AS order_id,
    c.name AS customer,
    p.name AS product,
    oi.quantity,
    oi.unit_price,
    (oi.quantity * oi.unit_price) AS line_total
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
ORDER BY o.id;
```

Explanation: This query combines four tables and creates a useful order report. line_total is calculated by multiplying quantity by unit price.

17. Aggregate Functions

COUNT

```
SELECT COUNT(*) AS total_products FROM products;
```

Explanation: Counts how many products exist.

AVG

```
SELECT AVG(price) AS average_price FROM products;
```

Explanation: Calculates the average product price.

MAX

```
SELECT MAX(price) AS highest_price FROM products;
```

Explanation: Returns the highest price.

MIN

```
SELECT MIN(price) AS lowest_price FROM products;
```

Explanation: Returns the lowest price.

SUM

```
SELECT SUM(total_amount) AS total_sales FROM orders WHERE status = 'Completed';
```

Explanation: Adds completed order totals.

18. GROUP BY

```
SELECT
    c.name AS category,
    COUNT(p.id) AS total_products
FROM categories c
LEFT JOIN products p ON c.id = p.category_id
GROUP BY c.id, c.name
ORDER BY total_products DESC;
```

Explanation: This counts how many products belong to each category. LEFT JOIN also keeps categories that currently have zero products.

19. HAVING

```
SELECT
    category_id,
    COUNT(*) AS total_products
FROM products
GROUP BY category_id
HAVING COUNT(*) >= 2;
```

Explanation: WHERE filters individual rows before grouping. HAVING filters the grouped result after GROUP BY.

20. Subquery Example

```
SELECT name, price
FROM products
WHERE price > (
    SELECT AVG(price) FROM products
);
```

Explanation: The inner query calculates the average price. The outer query returns products priced above that average.

21. Create an Index

```
CREATE INDEX idx_products_name ON products(name);
```

Explanation: An index can make searches faster when a table becomes large. Do not create unnecessary indexes because indexes also use storage and affect writes.

22. Create a View

```
CREATE VIEW product_details AS
SELECT
  p.id,
  p.name AS product,
  c.name AS category,
  p.price,
  p.is_active
FROM products p
JOIN categories c ON p.category_id = c.id;
```

Explanation: A view saves a SELECT query as a reusable virtual table.

```
SELECT * FROM product_details;
```

Explanation: This reads data from the view just like a normal table.

23. Transaction Example

```
START TRANSACTION;

UPDATE products
SET price = price + 2
WHERE category_id = 3;

COMMIT;
```

Explanation: A transaction groups related changes. COMMIT saves the changes. If you need to cancel before committing, use ROLLBACK.

```
START TRANSACTION;

UPDATE products
SET price = price + 100
WHERE category_id = 3;

ROLLBACK;
```

Explanation: ROLLBACK cancels the changes made inside the current transaction.

End of MySQL Assignment & Notes